Question 5: Count the Work Done by a Loop (Lucky-Draw Ticket Verification)

Student name	Sk.Thohith
Register number	1912260555
GitHub repository	CSA02-<RegisterNumber>-Assignment2 (folder: Q5)
Link	https://github.com/shaikthohith5353-ship-it/CSA02-1912260555-Assignment-2
Compiler	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Compile command	`gcc -Wall -Wextra -O2 -o q5_complete q5_complete.c`
Marks / Bloom level	7 marks / K4 – Analyse

1. Problem and design starting point

Prize tickets are the prime numbers. The first version tests every divisor from 2 to n−1. The organisers will verify tickets up to 1,000,003, so the number of divisions is the quantity that must be measured and reduced. Operation counts (divisions) are the primary evidence in this report; every number below was produced by the programs in the appendix.

1.1 From the principle to a loop condition

Principle given: *if n has a divisor greater than √n, it must also have a divisor smaller than √n.* Write a composite $n = a \times b$ with $a \le b$. Then $a \times a \le a \times b = n$, so the smaller factor always satisfies $a^2 \le n$. Therefore, if no i with $i^2 \le n$ divides n, no divisor exists at all and n is prime.

Question	Answer derived
Loop condition without calling sqrt()	`i * i <= n` (overflow-safe equivalent: `i <= n / i`, because for positive integers $i \le \lfloor n/i \rfloor \Leftrightarrow i^2 \le n$)
Where break / return stops the loop early	At the **first** i with `n % i == 0`: return 0 (composite) or set prime = 0 and break. If the loop ends normally, no divisor was found: return 1.
How the divisions are counted	A counter `checks++` placed immediately before each `n % i` test; it is reset to 0 at the start of every call.

1.2 Predictions before execution

For a prime n the improved loop tests $i = 2 \ldots \lfloor \sqrt{n} \rfloor$, i.e. $\lfloor \sqrt{n} \rfloor - 1$ divisions; the original loop tests $n - 2$ divisions. An even n stops at the very first division ($i = 2$).

n	Type	Predicted (i*i <= n)	Predicted (i < n)
97	prime	$\lfloor 9.85 \rfloor - 1 = 8$	$97 - 2 = 95$
100	even	1	1
7919	prime	$\lfloor 88.99 \rfloor - 1 = 87$	$7919 - 2 = 7917$
1000003	prime	$\lfloor 1000.0015 \rfloor - 1 = 999$	$1000003 - 2 = 1000001$

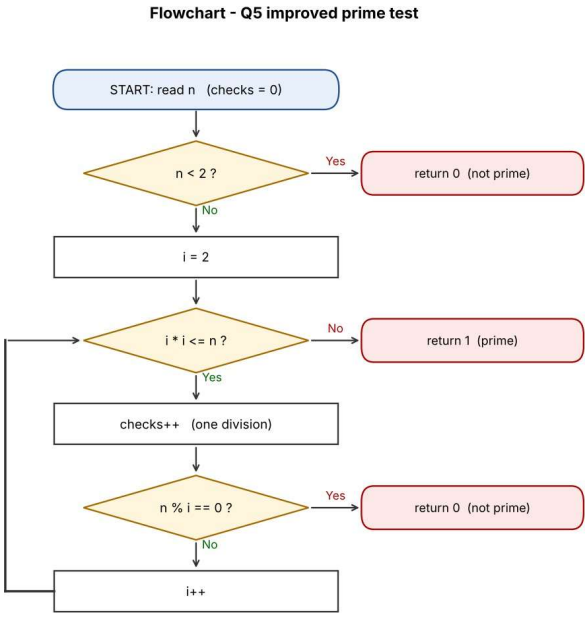


*Figure 1: flowchart of the improved prime test (condition i*i <= n, early return)*

2. Complete the Code (TODO 1)

Only the body marked TODO 1 was replaced; the rest of the program is unchanged.

```
#include <stdio.h>
unsigned long checks = 0; /* number of divisions performed by the last call */
/* Returns 1 if n is prime, otherwise 0. */
int is_prime(unsigned long n)
{
checks = 0;
if (n < 2) return 0;
for (unsigned long i = 2; i * i <= n; i++) {
    checks++;
    if (n % i == 0) return 0;
}
return 1;
}
int main(void)
{
unsigned long tickets[] = {97, 100, 7919, 1000003};
for (int k = 0; k < 4; k++) {
int p = is_prime(tickets[k]);
printf("%8lu %-9s divisions = %lu\n", tickets[k],
p ? "prime" : "not prime", checks);
}
return 0;
}
```

Logic. The loop runs while `i * i <= n`. *checks* is incremented before every division, so it equals the number of `%` operations. As soon as `n % i == 0` the function returns 0; if the loop condition fails first, no divisor $\leq \sqrt{n}$ exists and 1 is returned. Because the three-part `for` header holds the whole loop control, there is no separate flag.

2.1 Expected versus observed output (gcc 13.3.0)

Ticket	Expected output	Observed output	Match
97	prime, divisions = 8	prime, divisions = 8	yes
100	not prime, divisions = 1	not prime, divisions = 1	yes
7919	prime, divisions = 87	prime, divisions = 87	yes
1000003	prime, divisions = 999	prime, divisions = 999	yes

```
      97 prime     divisions = 8
     100 not prime divisions = 1
```

```
   7919 prime     divisions = 87
1000003 prime     divisions = 999
```

2.2 Effect of changing the condition to i < n

Using the same program with the condition replaced by `i < n` (file q5_complete_i_lt_n.c):

```
     97 prime     divisions = 95
    100 not prime divisions = 1
   7919 prime     divisions = 7917
1000003 prime     divisions = 1000001
```

The answers are identical (prime / not prime) but the work for the prime tickets grows from 8 → 95, 87 → 7,917 and 999 → 1,000,001. For the even ticket 100 nothing changes (1 division), because the early return fires at i = 2 in both versions. This shows that the *result* does not depend on the boundary, while the *cost* does.

3. Three loop forms and the division counts

The improved test was written three times with the same condition: `for`, `while` and `do-while` (source in Appendix A, function names prime_for / prime_while / prime_dowhile). Division counts for eight tickets, including a large prime (1000003), the largest prime below 10^6 (999983), a large composite and the special cases 2 and 3:

n	Result	2..n−1 (naive)	for	while	do-while
2	prime	0	0	0	0
3	prime	1	0	0	0
97	prime	95	8	8	8
100	composite	1	1	1	1
7919	prime	7,917	87	87	87
999983	prime	999,981	998	998	998
1000001 (=101×9901)	composite	100	100	100	100
1000003	prime	1,000,001	999	999	999

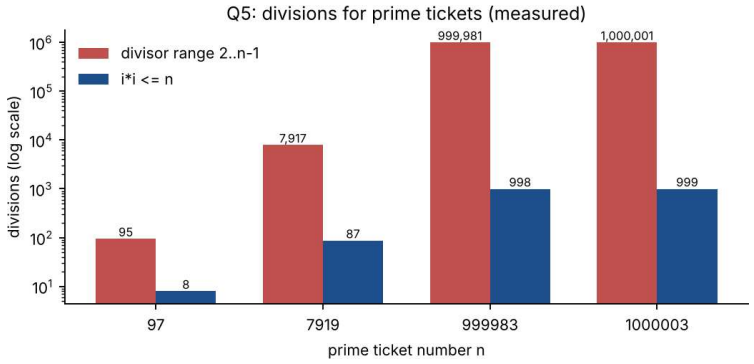


Figure 2: divisions for prime tickets, log scale. The gap is about 1000× at n = 10^6.

Observations. (1) The three loop constructs perform exactly the same number of divisions: the loop *form* does not change the work, the loop *condition* does. (2) For primes the improved count is $\lfloor\sqrt{n}\rfloor - 1$, matching the prediction table. (3) For composites the cost is set by the smallest factor (101 for 1000001), not by n. (4) Over every ticket 1 … 20,000 the naive loop performs 21,267,570 divisions and the improved loop 301,276 (about 70× fewer); over 1 … 1,000,003 the improved loop performs 67,741,504 divisions and all three constructs return identical answers for every n (0 mismatches).

4. Inquiry

4.1 Why the do-while version needs care for n = 2 and n = 3

A do-while executes its body **once before** the condition is examined. With i starting at 2 the body performs `n % 2` even though the principle says no division is needed when $i^2 = 4 > n$.

n	Unguarded do-while result	Divisions	Correct answer
2	**composite** (2 % 2 == 0)	1	prime — wrong
3	prime	1	prime (correct, but one needless division)
4	composite	1	composite
5	prime	1	prime (needless division)

Two correct remedies were tested: handle n < 4 before the loop (prime_dowhile in the appendix), or test the boundary at the top of the body `if (i > n / i) break;` as in the student program in Figure 4. Both give 0 divisions for 2 and 3 and identical answers for every n.

4.2 Why i * i <= n can overflow

If n and i are 32-bit unsigned values and n is close to 2^{32}, i^2 needs 33 bits once i = 65536. The product wraps modulo 2^{32} and the test becomes true again. Measured with n = 4,294,967,291 (the largest prime below 2^{32}):

```
i=65534 i*i (32-bit) = 4294705156 (i*i<=n is TRUE)
i=65535  i*i (32-bit) = 4294836225  (i*i<=n is TRUE)
i=65536 i*i  (32-bit)  =  0        (i*i<=n is TRUE)   <-- wrapped, loop should have stopped
i=65537 i*i (32-bit) = 131073      (i*i<=n is TRUE)
true 65536*65536 = 4294967296 (needs 33 bits)
Safe form i <= n/i: at i = 65536, n/i = 65535 -> condition FALSE (stops correctly)
```

After the wrap the loop no longer stops at $\sqrt{n}$; it can only end when some i divides n. For a prime this happens at i = n, so the loop would run about 4.3 billion divisions and report "not prime" (reasoned from the measured wrap; the full run was not executed). With signed int the overflow is undefined behaviour. In the assignment program `unsigned long` is 64-bit ($i^2 \le 1{,}000{,}003$ is safe), but the form `i <= n / i` is overflow-free for every n and costs the same number of divisions.

5. Alternative concept: Sieve of Eratosthenes

How the solution changes. Instead of testing each ticket separately, all numbers up to N are classified once. An array of N+1 flags is initialised to 1; for each prime i with $i^2 \le N$ all multiples from i^2 upward are set to 0. Afterwards each ticket is verified by one array lookup.

```
sieve(N):
    flag[0..N] = 1 ;  flag[0] = flag[1] = 0
    for i = 2 while i*i <= N:
        if flag[i] == 1:
            for j = i*i ; j <= N ; j += i:   flag[j] = 0

is_prime_query(n):   return flag[n]              /* O(1) */
```

Measured on N = 1,000,003 (file q5_sieve.c, same machine):

```
Sieve N=1000003: inner-loop writes=2122051, outer iterations=999, primes found=78499, time=0.0031 s,
    memory=1000004 bytes
Trial division for all n<=N: total divisions=67741504, mismatches vs sieve=0, time=0.2475 s
Average divisions per query (n uniform in 0..N) = 67.74
Break-even number of queries ~ 31326 (sieve writes / avg divisions)
1000 random tickets: trial divisions=64779, sieve lookups=1000
```

	Trial division (per ticket)	Sieve of Eratosthenes
Time, one query	$O(\sqrt{n})$ – 999 divisions for n = 1000003; O(1) for even n	O(1) lookup (after the sieve is built)
Time, preparation	none	O(N log log N): 2,122,051 writes measured for N = 1,000,003
Time, all N numbers	$\approx O(N\sqrt{N})$ worst case; measured 67,741,504 divisions	O(N log log N)
Space	O(1)	O(N): 1,000,004 bytes (one byte per flag; N/8 bytes if bit-packed)
Measured, whole range 0..N	0.245 s	0.0024 s (build) – results agree, 0 mismatches

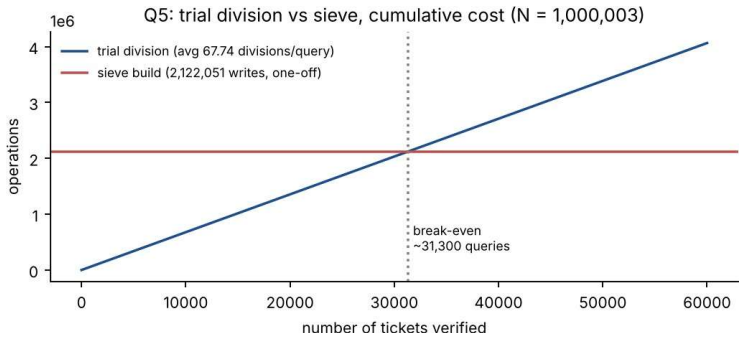


Figure 3: cumulative cost. The sieve pays off after about 31,300 randomly chosen queries (computed from the measured averages).

Trade-offs. For 1,000 random tickets trial division needs 64,779 divisions in total, which is *less* than the 2.1 million writes needed to build the sieve; for a few thousand verifications or an unknown, unbounded n, trial division is cheaper and needs no memory. When tickets are verified in bulk (tens of thousands of queries, or every number in a range) the sieve is far cheaper per query. The sieve also needs the upper limit N to be known in advance and memory proportional to N.

6. Student programs and execution evidence

The while and do-while programs below were executed by the student; their screenshots are reproduced as submitted. I re-compiled the same sources with `gcc -Wall` and tested n = 0, 1, 2, 3, 4, 9, 17, 19, 25, 97, 7919: every answer was correct, including n = 2 and 3, because the boundary `i <= n / i` is tested before the first division.

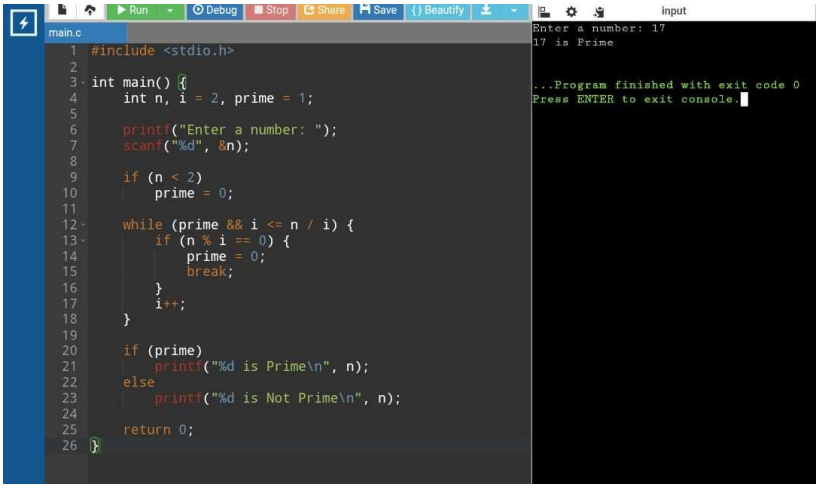


Figure 4a: while-loop version (condition: prime && i <= n / i), input 17

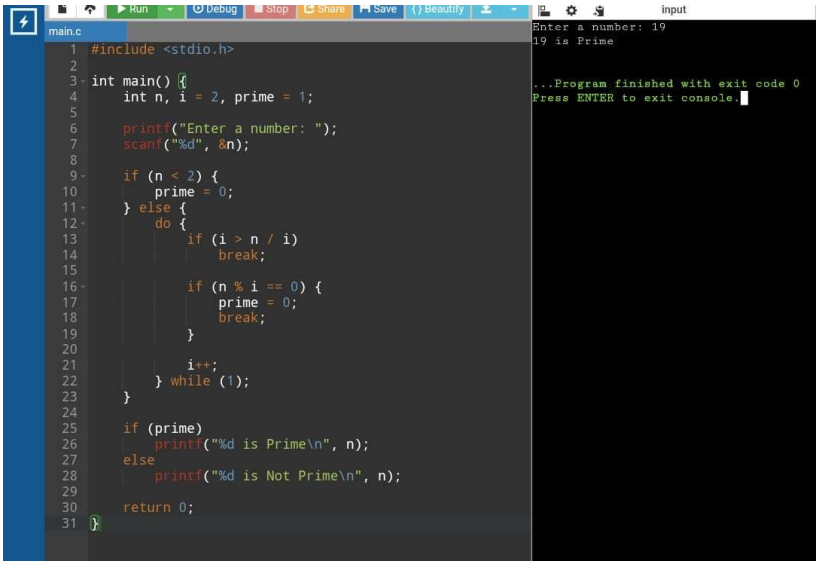


Figure 4b: do-while version (boundary test at the top of the body), input 19

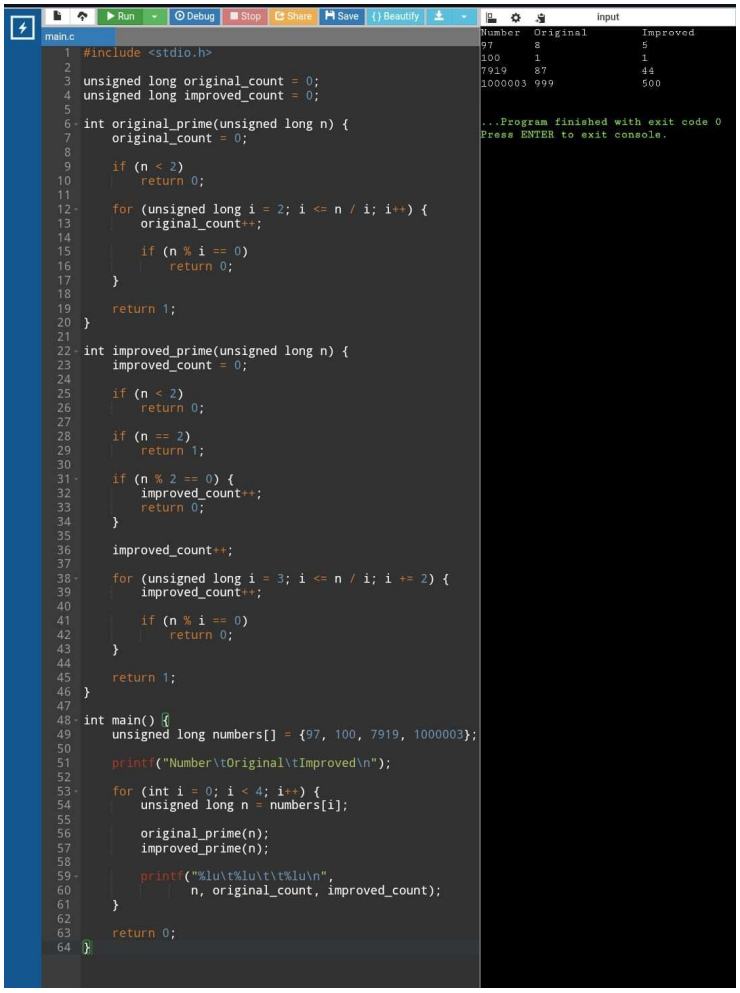


Figure 5: comparison program. 'Original' = improved loop (8, 1, 87, 999); 'Improved' = odd divisors only (5, 1, 44, 500).

7. Test table (normal, boundary and special cases)

Input n	Reason for the test	Expected	Observed	Divisions (i*i<=n)
0, 1	below 2	not prime	not prime	0
2, 3	smallest primes; do-while hazard	prime	prime	0
4	smallest composite	not prime	not prime	1
9, 25, 49	squares of primes (i = $\sqrt{n}$ boundary)	not prime	not prime	2 / 4 / 6
97	prime, spec value	prime	prime	8
100	even	not prime	not prime	1
7919	prime, spec value	prime	prime	87
999983	largest prime below 10^6	prime	prime	998
1000001	composite with large smallest factor	not prime	not prime	100
1000003	large prime, spec value	prime	prime	999

The squares 9, 25 and 49 confirm that the condition must be i*i <= n (with equality): with `i * i < n` the divisor i = $\sqrt{n}$ would be missed and these composites would be reported as prime.

8. Conclusion

Deriving the loop from the $\sqrt{n}$ principle reduces the division count for the large prime 1000003 from 1,000,001 to 999 without changing any answer. The loop construct is a matter of style; the boundary condition, the early return, the do-while guard and the overflow-safe form of the test are what decide correctness and cost. For one-off checks the O($\sqrt{n}$) test is preferred; for bulk verification the O(N log log N) sieve with O(N) memory is preferred.

Appendix A: source code (all executed with gcc 13.3.0)

q5_versions.c - naive, for, while, do-while, do-while without guard, overflow demo, totals

```
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

unsigned long checks;

/* V1: original version, divisors 2 .. n-1 (for) */
int prime_naive(unsigned long n){
    checks = 0; if (n < 2) return 0;
    for (unsigned long i = 2; i < n; i++) { checks++; if (n % i == 0) return 0; }
    return 1;
}
/* V2: improved, for loop, condition i <= n / i (no sqrt, no overflow) */
int prime_for(unsigned long n){
    checks = 0; if (n < 2) return 0;
    for (unsigned long i = 2; i <= n / i; i++) { checks++; if (n % i == 0) return 0; }
    return 1;
}
/* V3: improved, while loop */
int prime_while(unsigned long n){
    checks = 0; if (n < 2) return 0;
    unsigned long i = 2;
    while (i <= n / i) { checks++; if (n % i == 0) return 0; i++; }
    return 1;
}
/* V4: improved, do-while loop (needs the n < 4 guard) */
int prime_dowhile(unsigned long n){
    checks = 0; if (n < 2) return 0;
    if (n < 4) return 1;                /* n = 2, 3 handled before the loop */
    unsigned long i = 2;
    do { checks++; if (n % i == 0) return 0; i++; } while (i <= n / i);
    return 1;
}
/* V4-bug: do-while WITHOUT the guard */
int prime_dowhile_unguarded(unsigned long n){
    checks = 0; if (n < 2) return 0;
    unsigned long i = 2;
    do { checks++; if (n % i == 0) return 0; i++; } while (i * i <= n);
    return 1;
}

int main(void){
    unsigned long t[] = {2, 3, 97, 100, 7919, 999983, 1000001, 1000003};
    int nt = sizeof t / sizeof t[0];
    printf("== Division counts (checks) ==\n");
    printf("%-9s %-9s %10s %10s %10s %10s\n","n","result","naive","for","while","do-while");
    for (int k = 0; k < nt; k++){
        unsigned long n = t[k]; unsigned long c[4]; int r[4];
        r[0]=prime_naive(n); c[0]=checks;
        r[1]=prime_for(n); c[1]=checks;
        r[2]=prime_while(n); c[2]=checks;
        r[3]=prime_dowhile(n); c[3]=checks;
        int same = (r[0]==r[1] && r[1]==r[2] && r[2]==r[3]);
        printf("%-9lu %-9s %10lu %10lu %10lu %10lu %s\n", n, r[0]?"prime":"composite",
               c[0],c[1],c[2],c[3], same?"":"MISMATCH");
    }
    printf("\n== do-while without guard ==\n");
    for (unsigned long n = 2; n <= 5; n++){
        int r = prime_dowhile_unguarded(n);
        printf("n=%lu -> %s (divisions=%lu) [correct: %s]\n", n, r?"prime":"composite", checks,
               (n==2||n==3||n==5)?"prime":"composite");
    }
    printf("\n== 32-bit overflow of i*i <= n ==\n");
    {
        uint32_t n = 4294967291u; /* largest prime below 2^32 */
        uint32_t i; unsigned long iters = 0;
        for (i = 65534; iters < 6; i++, iters++){
            uint32_t sq = i * i;
            printf("i=%u  i*i (32-bit) = %u  (i*i<=n is %s)\n", i, sq, (sq <= n)?"TRUE":"FALSE");
        }
        printf("true i*i for 65536 = %llu (needs 33 bits)\n", 65536ULL*65536ULL);
        printf("Safe form i <= n/i: at i=65536, n/i = %u -> condition %s\n", n/65536u,
               (65536u <= n/65536u)?"TRUE":"FALSE");
    }
    printf("\n== Totals for every n = 1..1000003 ==\n");
    {
```

```
        unsigned long N = 1000003, tot_for = 0, tot_while=0, tot_do=0, bad=0;
        for (unsigned long n = 1; n <= N; n++){
            int a = prime_for(n); tot_for += checks;
            int b = prime_while(n); tot_while += checks;
            int c = prime_dowhile(n); tot_do += checks;
            if (a!=b || b!=c) bad++;
        }
        printf("total divisions: for=%lu while=%lu do-while=%lu, mismatches=%lu\n",
            tot_for,tot_while,tot_do,bad);
        unsigned long tot_naive = 0;
        for(unsigned long n = 1; n <= 20000; n++){ prime_naive(n); tot_naive += checks; }
        unsigned long tot_for20k = 0;
        for (unsigned long n = 1; n <= 20000; n++){ prime_for(n); tot_for20k += checks; }
        printf("n = 1..20000: naive total=%lu, sqrt total=%lu\n", tot_naive, tot_for20k);
    }
    return 0;
}
```

Output of q5_versions

```
== Division counts (checks) ==
n          result          naive        for       while    do-while
2          prime               0          0           0           0
3          prime               1          0           0           0
97         prime              95          8           8           8
100        composite           1          1           1           1
7919       prime            7917         87          87          87
999983     prime          999981        998         998         998
1000001    composite         100        100         100         100
1000003    prime          1000001        999         999         999

== do-while without guard ==
n=2 -> composite (divisions=1) [correct: prime]
n=3 -> prime (divisions=1) [correct: prime]
n=4 -> composite (divisions=1) [correct: composite]
n=5 -> prime (divisions=1) [correct: prime]

== 32-bit overflow of i*i <= n ==
i=65534  i*i (32-bit) = 4294705156  (i*i<=n is TRUE)
i=65535  i*i (32-bit) = 4294836225  (i*i<=n is TRUE)
i=65536  i*i (32-bit) = 0  (i*i<=n is TRUE)
i=65537  i*i (32-bit) = 131073  (i*i<=n is TRUE)
i=65538  i*i (32-bit) = 262148  (i*i<=n is TRUE)
i=65539  i*i (32-bit) = 393225  (i*i<=n is TRUE)
true i*i for 65536 = 4294967296 (needs 33 bits)
Safe form i <= n/i: at i=65536, n/i = 65535 -> condition FALSE

== Totals for every n = 1..1000003 ==
total divisions: for=67741504 while=67741504 do-while=67741504, mismatches=0
n = 1..20000: naive total=21267570, sqrt total=301276
```

q5_sieve.c – sieve of Eratosthenes versus trial division

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
unsigned long checks;
int prime_for(unsigned long n){
    checks = 0; if (n < 2) return 0;
    for (unsigned long i = 2; i <= n / i; i++) { checks++; if (n % i == 0) return 0; }
    return 1;
}
static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec + t.tv_nsec*1e-9;
     }
int main(void){
    const unsigned long N = 1000003;
    unsigned char *flag = malloc(N + 1);
    unsigned long marks = 0, scan = 0;
    double t0 = now();
    for (unsigned long i = 0; i <= N; i++) flag[i] = 1;
    flag[0] = flag[1] = 0;
    for (unsigned long i = 2; i * i <= N; i++){
        scan++;
        if (flag[i]) for (unsigned long j = i * i; j <= N; j += i){ flag[j] = 0; marks++; }
    }
    double t_sieve = now() - t0;
    unsigned long primes = 0; for (unsigned long i = 0; i <= N; i++) primes += flag[i];
    printf("Sieve N=%lu: inner-loop writes=%lu, outer iterations=%lu, primes found=%lu, time=%.4f s, memory=%lu
        bytes\n",
           N, marks, scan, primes, t_sieve, N + 1);
    /* correctness + cost of trial division for every n <= N */
    unsigned long tot = 0, bad = 0;
    t0 = now();
    for (unsigned long n = 0; n <= N; n++){ int r = prime_for(n); tot += checks; if (r != flag[n]) bad++; }
    double t_trial = now() - t0;
    printf("Trial division for all n<=N: total divisions=%lu, mismatches vs sieve=%lu, time=%.4f s\n", tot, bad,
         t_trial);
    printf("Average divisions per query (n uniform in 0..N) = %.2f\n", (double)tot/(N+1));
    printf("Break-even number of queries ~ %.0f (sieve writes / avg divisions)\n",
        (double)marks/((double)tot/(N+1)));
    /*  1000  sample  tickets  */
    srand(12345); unsigned long q1000 = 0;
    for (int k = 0; k < 1000; k++){ unsigned long n = 2 + (unsigned long)rand() % (N - 1); prime_for(n); q1000
        += checks; }
    printf("1000 random tickets: trial divisions=%lu, sieve lookups=1000\n", q1000);
    free(flag); return 0;
}
```